

Below are the **6 project-related interview questions** from your uploaded PDF, answered **specifically for your Flight Passenger Satisfaction Prediction Project**, using details from your **app.py**, **model pipeline**, and overall workflow.

All answers follow the **structure and guidelines shown in the PDF** and are customized to your project.

✅ 1. Explain about your project (Using STAR Approach)

Project based questions

+

app

Situation

I worked on a **Flight Passenger Satisfaction Prediction System**, aiming to predict whether a passenger is *Satisfied*, *Neutral*, or *Dissatisfied* based on flight experience factors such as onboard service, delay time, cleanliness, seat comfort, and more.

Task

My role was to handle the full **ML pipeline**, including data preprocessing, model building, model saving, and developing a **Streamlit web application** to make real-time predictions.

Action

- Performed data cleaning, handled categorical features, and scaled the necessary attributes.
- Trained multiple ML models and finally selected a **Random Forest Pipeline**, stored as "rf_pipeline.joblib".
- Used **LabelEncoder** (stored as "label_encoder_target.joblib") to convert prediction outputs back into readable labels.
- Built a **Streamlit UI (app.py)** that takes user inputs like: gender, customer type, class, delays, onboard service ratings, entertainment, food, legroom comfort etc.
- Integrated the model so that clicking the **Predict Satisfaction** button generates a label such as *Satisfied / Neutral / Dissatisfied*.

Result

The project resulted in a **fully functional predictive web app** that helps airlines quickly assess passenger satisfaction trends. It automates classification and makes decision-making faster, leading to improved service strategies and customer experience.

✓ 2. What challenges did you face in your project, and how did you overcome them?

Project based questions

1 Handling Categorical Data Properly

Challenge:

The dataset had many categorical columns like **Gender, Customer Type, Travel Type, Class**, etc. Encoding them incorrectly was reducing accuracy.

Solution:

I used **One-Hot Encoding** and **Label Encoding** inside a **Pipeline**, ensuring preprocessing happened consistently during training and prediction.

2 Imbalanced Classes (Satisfied vs Neutral vs Dissatisfied)

Challenge:

The distribution of labels was uneven, causing biased predictions.

Solution:

I applied **class balancing techniques** and experimented with algorithms like Random Forest, which naturally handle imbalance better.

3 Feature Scaling for Delay & Distance Values

Challenge:

Delay values ranged from 0 to 1000 minutes, while survey ratings ranged only from 0–5.

Solution:

Used scaling inside the pipeline so the model would treat all features uniformly.

4 Deployment Issues in Streamlit

Challenge:

Loading the model (`rf_pipeline.joblib`) and encoder (`label_encoder_target.joblib`) inside Streamlit caused repeated loading and speed issues.

Solution:

Used **@st.cache_resource** (as seen in your `app.py`) to load them once efficiently.

✓ Faster predictions

✓ No redundant model loading

✓ 3. What technologies/tools did you use, and why?

Project based questions

+

app

Programming & ML

Technology	Reason
------------	--------

Python	Simple, widely used for ML
--------	----------------------------

Pandas & NumPy	Efficient for cleaning and preprocessing
----------------	--

Scikit-Learn	Provided pipelines, encoders, and Random Forest implementation
--------------	--

Joblib	To save & load the model and encoders easily
--------	--

Model

- **Random Forest Classifier Pipeline**
Chosen because it handles mixed numerical/categorical data well and gives good accuracy.

Deployment

Tool	Reason
------	--------

Streamlit	Fastest way to create a ML web app; no HTML/CSS required
-----------	--

Slider & Selectbox UI widgets	Makes input collection intuitive for users
-------------------------------	--

Why Streamlit?

- Rapid prototyping
- Clean UI
- Easy integration with Python models
- Very beginner-friendly for ML deployment

✓ 4. How did you ensure the project was completed successfully?

Project based questions

Clear Planning

I divided the entire project into phases:

Dataset → Preprocessing → EDA → Model Training → Evaluation → Deployment → Testing.

Iterative Model Improvement

Trained multiple models (Logistic Regression, Decision Tree, Random Forest) and selected the best-performing one.

Continuous Testing

- Validated predictions on real sample data.
- Ensured UI accepted correct data types.
- Verified the joblib model produced consistent results.

Version Control & Documentation

Maintained structured code and proper commenting so the Streamlit app and model were easy to maintain.

User-Centric Deployment

Built a clean UI that airline staff could use easily without ML knowledge.

5. What impact did your project have? / Real-world impact / How will it help our company?

Project based questions

Real-World Impact

- Helps airlines understand **why passengers are unsatisfied**.
- Enables **data-driven decisions** to improve onboard service, customer handling, and operational efficiency.
- Predicts satisfaction without requiring manual survey reviews.

Business Value

- Reduces customer churn by identifying dissatisfaction patterns early.
- Improves Net Promoter Score (NPS).
- Helps optimize resource allocation (food, entertainment, seating improvements).

How These Skills Help Your Company

Your skills align with industry expectations:

- ✓ Machine Learning model development
- ✓ Data preprocessing & feature engineering
- ✓ Deploying ML apps for business use
- ✓ Building real-time dashboards (Streamlit UI)

You can directly support roles like:

- Data Analyst
 - ML Engineer
 - AI Developer
 - Business Intelligence Engineer
-

✓ 6. Why did you use this tech stack? Explain its advantages.

Project based questions

Why This Tech Stack Was Ideal

Python

- Most widely used ML language
- Huge ecosystem of libraries

Pandas & NumPy

- Efficient for manipulating large datasets
- Easy handling of mixed data types (survey + numeric fields)

Scikit-Learn

- Provides everything needed:
 - ✓ Preprocessing
 - ✓ Pipelines
 - ✓ Classification models
- Stable & beginner-friendly

Random Forest

- Handles non-linear relationships
- Works well with categorical + numerical features

- Avoids overfitting better than many algorithms

Streamlit

- Best tool to convert ML model into a working app
- Requires zero frontend skills
- Clean UI using simple widgets like sliders and select boxes

Joblib

- Light-weight model saving
- Fast loading inside the app